面向边缘智能的多范式神经网络 FPGA 加速平台

摘要

边缘智能设备往往要同时处理图像、文本和时序信号，而 CNN、SNN、LNN 三类神经网络的计算形态差异很大：CNN 以密集卷积为主，SNN 以稀疏脉冲与事件驱动为主，LNN 以连续时间状态更新为主，三者的数值格式、存储访问与调度方式各不相同。传统做法是为每个模型单独设计一套加速器，模型到板卡的链路彼此割裂，仿真、时序与上板结果也难以复核，二次开发与迁移成本高。本作品在同一颗 AMD ZU15EG（xczu15eg-ffvb1156-2-i）上完成了三个神经网络加速库，并统一了从模型到板卡的方法。CNN 库把静态 INT8 QDQ ONNX 图转译为整图指令流、参数包、DDR 初始化镜像与模型专用 RTL，并输出 Vivado IP Catalog 源码包；支持 16 个操作码、11 类算子核，随附 MNIST、CIFAR-10、Depthwise V2、MaxPool2D 与 YOLOv8n-ReLU VOC20 五个案例，历史 r4 实现在 5 ns 下收敛并完成固定样本板测。SNN 库把配置与整数 checkpoint 转译为参数包、RTL 配置与片上调度，在 ZU15EG 上闭环了 Spikformer 中文情感分类：冻结 RTL 回放 0 mismatch、5 ns 时序收敛、平台 r7 位流生成，板端记录 3/3 样本匹配，整数精度为 dev 0.7592、holdout 0.7562。LNN 库把 CfC 液态神经网络单步计算图映射为 8 个可综合 RTL 模块，训练权重经单步 ONNX 与 int16 Q12 定点导出后直接成为 RTL 的 ROM 初值，64 步推理 9088 拍、约 30.3 μs，300 MHz 下时序收敛。三个库共用同一套“模型—数值合同—转译—仿真—时序—上板—证据”流程，每次转译生成绑定模型结构、参数清单与 RTL 源码哈希的 manifest，全部结果按 pass / fail / partial / not_run 分级记录，冲突数字并列来源，任一结论都可回溯到证据文件。研发过程由大模型智能体协作完成，沉淀出任务书、交接规范、证据纪律与可复用技能包，使他人能够按文档从零复现本作品。

第一部分 作品概述

1.1 功能与特性

（1）CNN 库：输入为静态 INT8 QDQ ONNX 图与数值配置，输出整图指令流、打包参数、DDR 初始化镜像、模型专用 RTL 顶层与 Vivado IP Catalog

源码包。支持 16 个操作码（0x0001–0x001D，另含候选 0x001E）与 11 类算子核，覆盖卷积、深度卷积、池化、Resize、Concat、LUT 激活等；随附 MNIST、CIFAR-10、Depthwise V2、MaxPool2D 与 YOLOv8n-ReLU VOC20 五个案例，并提供 ABI 1.0 寄存器与存储协议，模型更换只需重新编译，不改算子库。

（2）SNN 库：输入配置 JSON 与真实语料训练得到的整数 checkpoint，0.7 s 转译出 54 个参数 blob、RTL 结构配置、目标顶层、约束与片上调度；支持 Spikformer 结构参数化（E、H、时间步、序列与深度可配）与 8 类脉冲算子，实现中文评论情感分类；板上输入句子直接输出正面或负面，并提供 PYNQ 交互演示与三样本逐通道比对两种模式。

（3）LNN 库：输入训练权重与配置，经单步 ONNX 与 int16 Q12 定点导出，生成 RTL 权重 ROM 初值、激活查表与参数头文件；在 PL 上连续完成 64 步推理，每步 142 拍、共 9088 拍；换任务只需重训权重并重新导出，RTL 与位流结构不变。

1.2 应用领域

（1）边缘视觉：面向工业质检、安防监控与嵌入式视觉终端。以 YOLOv8n 为例，量化 QDQ ONNX 一次转译即可生成指令流、参数与 IP，接入不同板卡的 Block Design 后可部署；MobileDW-32 面向 CIFAR-10 级轻量分类，适合低功耗、低成本的现场识别。FPGA 侧按模型裁剪引擎，可在器件资源受限时保留主要算力。

（2）端侧中文文本：SNN 库以脉冲神经网络处理中文评论情感分析，全整数推理、无需联网，适合电商评论筛查、客服舆情初筛等隐私敏感场景；脉冲稀疏特性使计算以位与、popcount 与加法为主，便于在低功耗 FPGA 上部署。

（3）时序信号与控制：LNN 库的 CfC 单元适合连续时间序列，可扩展到传感器滤波、电机与生理信号预测等小规模时序建模任务；单步计算周期确定，便于硬实时控制与确定性延迟要求的场景。

1.3 主要技术特点

（1）统一转译与证据链：三个库共用“模型—数值合同—转译—仿真—时序—上板”流程；每次转译生成 manifest，绑定模型结构、参数清单与 RTL 源码哈希；结果按 pass / fail / partial / not_run 分级，任一 PASS 均可回溯到证据文件，降低多人（机）协作中的错配风险。

（2）数值合同先行：CNN 采用 affine-int8-v1 量化合同；SNN 为全整数 Spikformer 变体（LIF 33 位扩展、QK 位与加 popcount、AV 不归一、残差只加 0/1 脉冲）；LNN 为 int16 Q12 数据、int32 Q24 偏置、48 位精确累加与 1025 点 LUT；三处实现（Python 参考、寄存器模型、RTL）共用同一套舍入与饱和规则。

（3）资源与性能优化：CNN 按图裁剪引擎，YOLO 只实例化 5/13 个子引擎，平台 LUT 由 191,542 降到 140,028（−26.9%），DSP 由 527 降到 471（−10.6%）；SNN 以 5 级 MAC 流水换取 5 ns 收敛；LNN 用单 MAC 时分复用，CfC 核仅占 1 个 DSP48E2；16/8 并行度配置下 YOLO 板端端到端 51.4 ms（PL 约 31 ms + PS 等待约 20 ms）。

（4）软硬协同：PS 侧 PYNQ 以 MMIO 访问 AXI-Lite 寄存器、以 AXI-Full/DMA 搬运数据，PL 侧加速器通过 PL DDR4 与主机共享存储；桌面端脚本与板端运行共用同一份定点参考和同一套寄存器协议。

1.4 主要性能指标

主要指标如表 1 所示。资源与时序均取自仓库实测记录：CNN 为历史 r4 实现，SNN 为 5 ns OOC 与平台 r7，LNN 为最终位流实现；板端状态如实标注。

项目	CNN 库	SNN 库	LNN 库
代表模型	YOLOv8n-ReLU VOC20	Spikformer 情感分类	CfC 正弦延迟 4 步
目标器件 / 频率	xczu15eg-ffvb1156-2-i / 200 MHz	200 MHz（5 ns）	300 MHz
时序	WNS +0.075 ns / WHS +0.010 ns（历史 r4）	OOC WNS +0.135 / WHS +0.023；平台 r7 +0.051 / +0.011	WNS +0.226 ns / WHS +0.011 ns
片上资源	LUT 140,028；DSP 471；URAM 72（历史 r4 全平台）	LUT 40,964；FF 71,173；BRAM 186.5；DSP 128（OOC）	LUT 25,221；FF 32,890；DSP 4；BRAM 28.5
关键验证	103 条指令编译 + IP 打包；ModelSim 4 个案例；16/8 板端 51.4 ms	冻结 RTL 回放 0 mismatch；整数精度 dev 0.7592 / holdout 0.7562	64/64 逐位一致；定点 vs 浮点 MSE 1.17e-4

1.5 主要创新点

（1）提出“三范式一平台”的统一转译与证据方法：CNN、SNN、LNN 共用数值合同、源码哈希绑定与分级证据记录，换模型只重编译或重导出，RTL 库不动。

（2）CNN 采用编译期指令流加图感知引擎裁剪：未用引擎在生成阶段裁剪，平台 LUT 降低 26.9%、DSP 降低 10.6%，ABI 1.0 寄存器与存储协议保持不变。

（3）SNN 实现全整数 Spikformer 变体与 5 级 MAC 流水：5 ns 下 0 失败端点收敛，OOC 资源约 4 万 LUT、128 个 DSP，并生成可上板平台位流。

（4）LNN 将 CfC 单步图整单元映射为单 MAC 时分复用结构：CfC 核仅 1 个 DSP48E2，64 步周期确定（9088 拍），门控与查表语义在三处实现中保持一致。

1.6 设计流程

流程为：需求与模型选型 → 训练与量化（CNN 采用 QDQ 导出，SNN 采用真实语料训练与整数导出，LNN 采用浮点训练与单步 ONNX）→ 数值合同与转译 → 仿真验证（ModelSim、RTL 回放与整数参考）→ OOC 与平台实现 → 板端 PYNQ 部署 → 证据归档（manifest、报告与哈希）。大模型协作贯穿全流程：任务书拆解、交接文档与技能包保证每一步的输入、输出与验收口径明确。

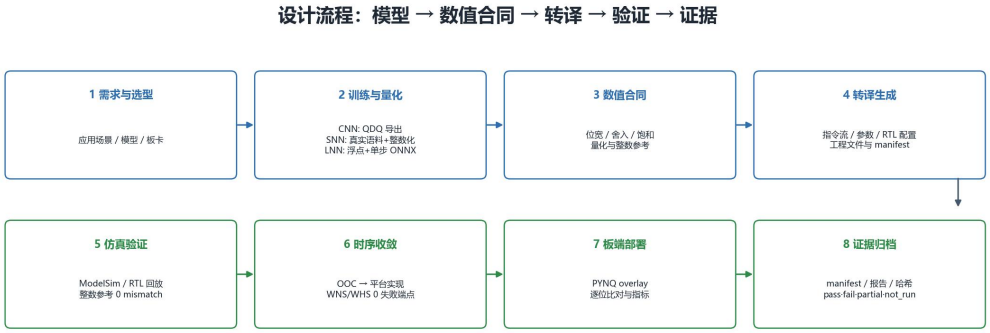


图 1 整体设计流程图

第二部分 系统组成及功能说明

2.1 整体介绍

系统由 PC 侧转译工具链与 ZU15EG 平台两侧组成。PC 侧负责模型训练与量化、数值合同、转译生成、仿真与时序门禁以及证据归档；平台侧由 PS（ARM Cortex-A53 四核 + PYNQ Linux）与 PL 组成。PS 通过 AXI-Lite 访问各加速器的寄存器窗口，完成任务配置、启动与状态轮询；通过 AXI-Full/DMA 与 PL DDR4 搬运参数与数据；PL 内并列部署 CNN、SNN、LNN 三个加速库，各自占用 AXI 主口与存储窗口。

三个库共用同一套交付形态：每次转译生成 manifest 与工程文件（参数、RTL 配置、约束、综合/实现入口），经仿真与时序门禁后生成位流，再由 PYNQ 驱动在板上运行并逐位比对；所有结果按证据路径归档，便于复核与复现。图 2 给出系统整体框图，图 6 给出软件工具链框图。

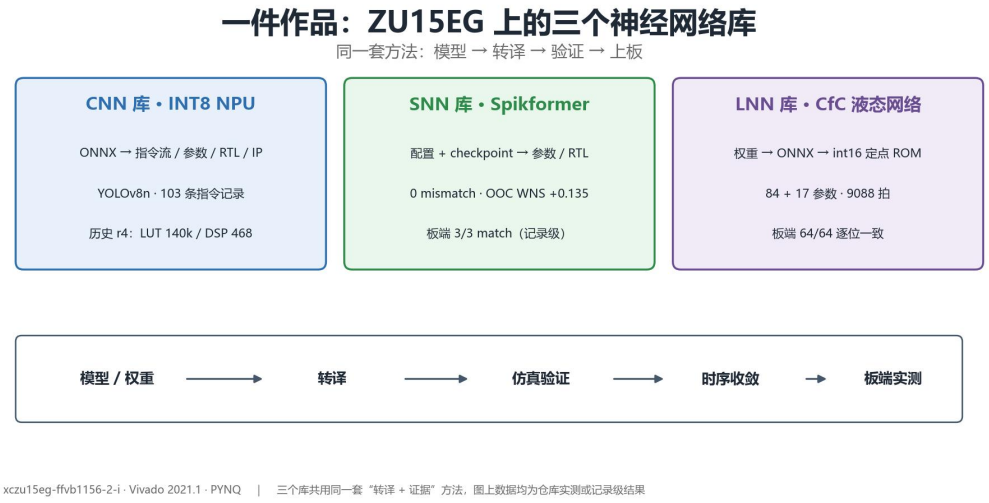


图 2 系统整体框图

2.2 硬件系统介绍

2.2.1 硬件整体介绍：硬件平台为 RIGUKE ZU15EG 开发板，核心器件为 AMD Zynq UltraScale+ MPSoC xczu15eg-ffvb1156-2-i；PS 侧为 ARM Cortex-A53 四核，运行 PYNQ Linux；PL 侧含 PL DDR4（1 GiB 可见窗口）、读/写 DMA 与 AXI-Lite 寄存器窗口；应用时钟按库分别为 200 MHz（CNN、SNN）与 300 MHz（LNN）。主要硬件配置如表 2。

项目	配置
开发板 / 器件	RIGUKE ZU15EG；xczu15eg-ffvb1156-2-i（Zynq UltraScale+ MPSoC）
PS	ARM Cortex-A53 ×4，PYNQ Linux（PYNQ 2.7）
PL DDR4	MIG 控制器，CPU 可见窗口 1 GiB（SNN / LNN 平台）
数据搬运	3 路读 DMA + 1 路写 DMA（SNN 平台）；CNN 采用 m_axi + PS DDR
时钟	CNN 200 MHz；SNN 200 MHz；LNN 300 MHz（单时钟域）
开发工具	Vivado 2021.1；ModelSim SE-64 2020.4（仿真）

2.2.2 机械设计介绍：本作品为 FPGA 加速器与软件工具链，无机械结构件，故无机械设计内容。

2.2.3 电路各模块介绍：FPGA 侧“电路”体现为平台 Block Design、约束与位流。以 SNN 平台为例，PS 的 M_AXI_HPM0_LPD 经 AXI-Lite SmartConnect 接到加速器 CSR，PS 的 S_AXI_HP0_FPD 经 SmartConnect 接 PL DDR4；LNN 平台由同一平台工程派生，npu_clk 为 300 MHz 单时钟域。地址映射与接口如表 3；SNN 平台框图见图 3，CNN 与 LNN 平台框图见图 4，约束与时序报告见图 5。

模块 / 接口	地址或参数	说明
加速器 CSR 窗口	0x4_8000_0000（4 KiB）	AXI4-Lite，32 位寄存器访问
PL DDR4 CPU 窗口	0x4_0000_0000（1 GiB）	PS 可见的 PL 侧存储窗口，DMA 与 MMIO 共用
CNN NPU CSR / DDR	0x8000_0000 / HP0 DDR	MobileDW PS-DDR 平台映射，与其他平台不可互换
约束	5 ns（200 MHz）/ 3.33 ns（300 MHz）	单时钟域，不降频、不加时序例外

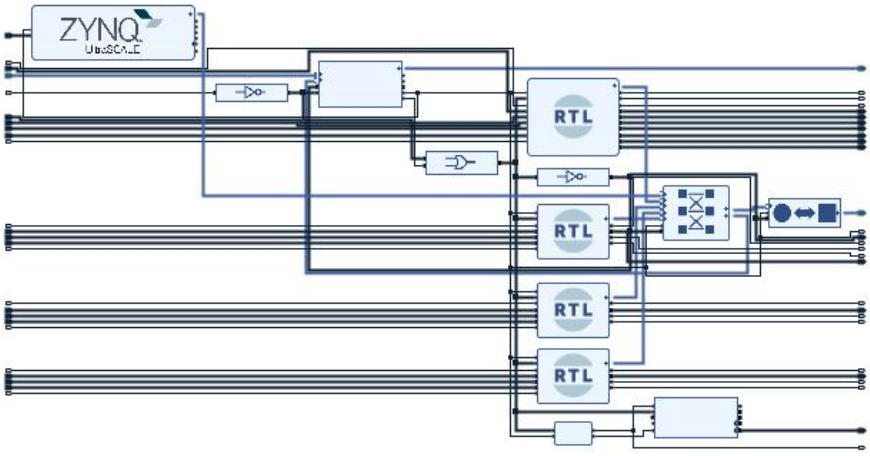
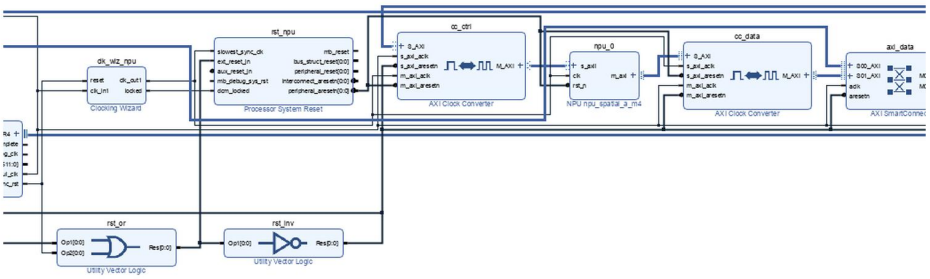


图 3 SNN 平台 Block Design（design_1）

CNN 平台（npu_0 + DMA + DDR4）



LNN 平台（design_1 + DMA）

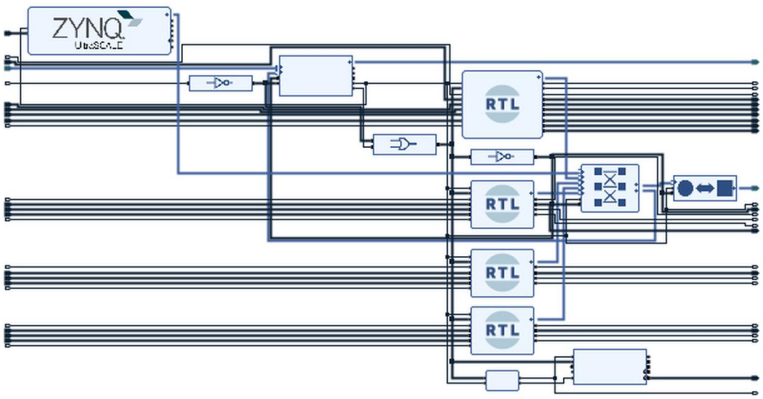


图 4 CNN 与 LNN 平台 Block Design

① D:/spk_platform/src/pin.xdc（DDR4 引脚约束）

D:/spk_platform/src/pin.xdc

```
64 set_property PACKAGE_PIN AM5  [get_ports C0_DDR4_0_reset_n]
65
66
67 set_property PACKAGE_PIN AH2  [get_ports {C0_DDR4_0_dm_n[0]}]
68 set_property PACKAGE_PIN AE5  [get_ports {C0_DDR4_0_dm_n[1]}]
69 set_property PACKAGE_PIN AH7  [get_ports {C0_DDR4_0_dm_n[2]}]
70 set_property PACKAGE_PIN AE10 [get_ports {C0_DDR4_0_dm_n[3]}]
71
72
73 set_property PACKAGE_PIN AH1  [get_ports {C0_DDR4_0_dqs_t[0]}]
74 set_property PACKAGE_PIN AJ1  [get_ports {C0_DDR4_0_dqs_c[0]}]
75 set_property PACKAGE_PIN AJ6  [get_ports {C0_DDR4_0_dqs_t[1]}]
76 set_property PACKAGE_PIN AJ5  [get_ports {C0_DDR4_0_dqs_c[1]}]
77 set_property PACKAGE_PIN AE8  [get_ports {C0_DDR4_0_dqs_t[2]}]
78 set_property PACKAGE_PIN AF8  [get_ports {C0_DDR4_0_dqs_c[2]}]
79 set_property PACKAGE_PIN AF11 [get_ports {C0_DDR4_0_dqs_t[3]}]
80 set_property PACKAGE_PIN AG11 [get_ports {C0_DDR4_0_dqs_c[3]}]
81
82
```

② Design Timing Summary（SNN 平台 r7：WNS +0.051 / WHS +0.011，0 失败端点）



图 5 约束文件与 Design Timing Summary（SNN 平台 r7）

2.3 软件系统介绍

2.3.1 软件整体介绍：PC 侧三个工具链分别对应三个库。CNN 工具链由 ONNX 前端（图解析、量化合同、图降低、规划、指令与 RTL 生成）与 IP 打包器组成；SNN 工具链提供 compile / build / package 三个子命令，完成参数打包、RTL 配置、调度与板端运行包生成；LNN 工具链完成 ONNX 导出、定点导出与 manifest 生成。板端软件为 PYNQ 运行时：加载 overlay、按寄存器协议配置并启动加速器、读回结果、与定点参考比对并落盘；三个库共用同一套“证据脚本”，把结果整理为 summary、报告与哈希清单；三库工程层次见图 7。

2.3.2 软件各模块介绍：各模块的输入、输出与职责如表 4。

模块	关键输入	关键输出	说明
CNN 编译器（tools/onnx_gen）	QDQ ONNX、数值配置、输入样本	图规划、指令流、参数包、DDR 镜像、模型专用 RTL	图降低与指令编码，源码哈希写入 manifest
CNN IP 打包器（tools/npu_gen）	生成的 RTL 与源清单	Vivado IP Catalog 源码包（component.xml）	不含 Block Design 与位流
SNN 工具链（tools/toolchain.py）	配置 JSON + 整数 checkpoint	54 个参数 blob、RTL 配置、调度、约束与 Tcl	compile / build / package 三段式
LNN 导出器（model/generate_export.py）	浮点权重与训练配置	int16 Q12 权重/偏置、LUT、RTL ROM 初始化头文件	与仓库导出逐字节一致
PYNQ 运行时（board/）	overlay、参数包、输入样本	输出比对结果、summary.json、日志与哈希	区分 hardware 与 simulation-model
验证与证据脚本	manifest、仿真/综合报告	分级门禁结果与证据索引	pass / fail / partial / not_run + 证据路径

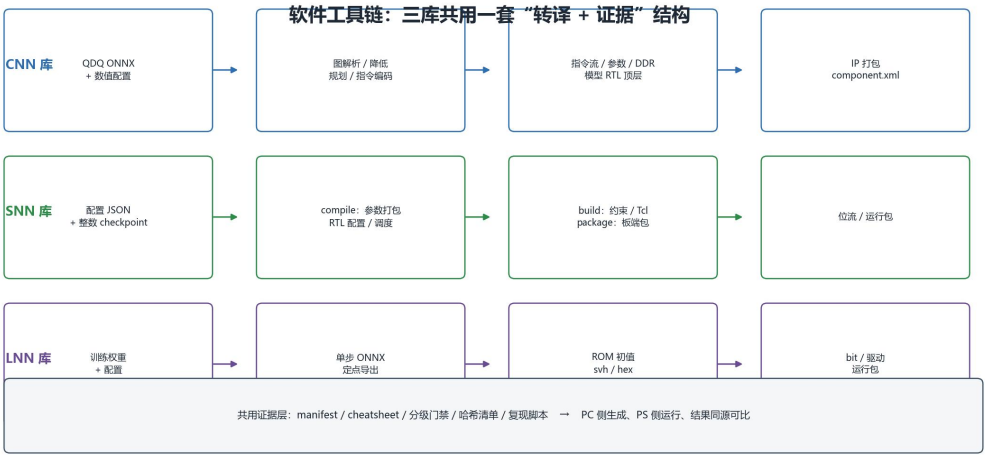


图 6 软件工具链框图

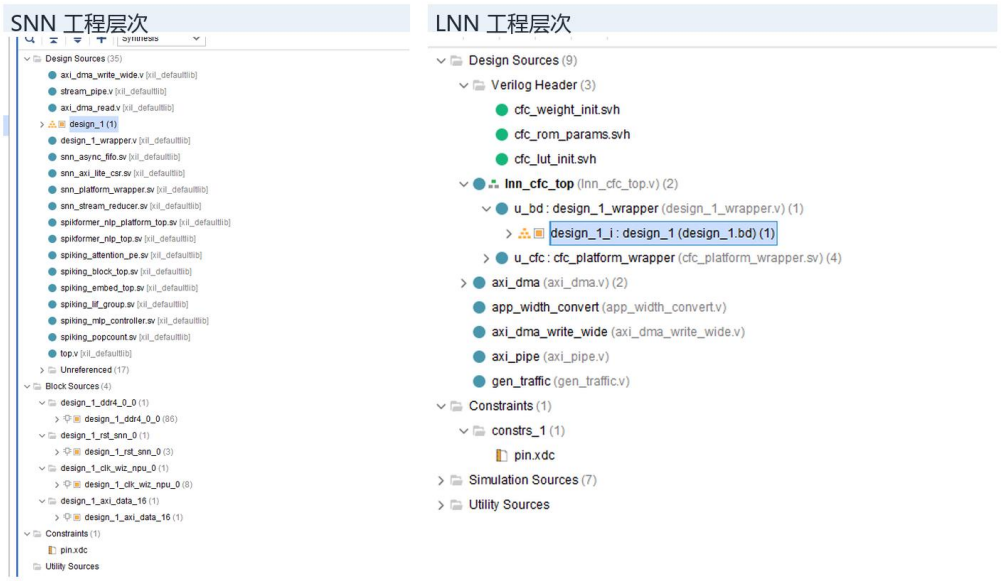


图 7 工程层次截图（左：SNN；右：LNN）

第三部分 完成情况及性能参数

3.1 整体介绍

系统按“转译生成—仿真验证—物理实现—板端部署”的分层方式完成。三个库完成度不同，均按证据如实标注：CNN 库工具链与 IP 生成已闭环，历史 r4 实现取得 5 ns 收敛并完成固定样本板测，16/8 并行度平台完成 YOLO 板端 51.4 ms 推理；SNN 库 NLP 主线全链闭环（回放 0 mismatch、OOC、IP、平台 r7 位流、板端记录）；LNN 库完成 300 MHz 位流、三件套哈希配对与板端离线模型自检。图 8、图 9 为实物与上板调试环境照片。

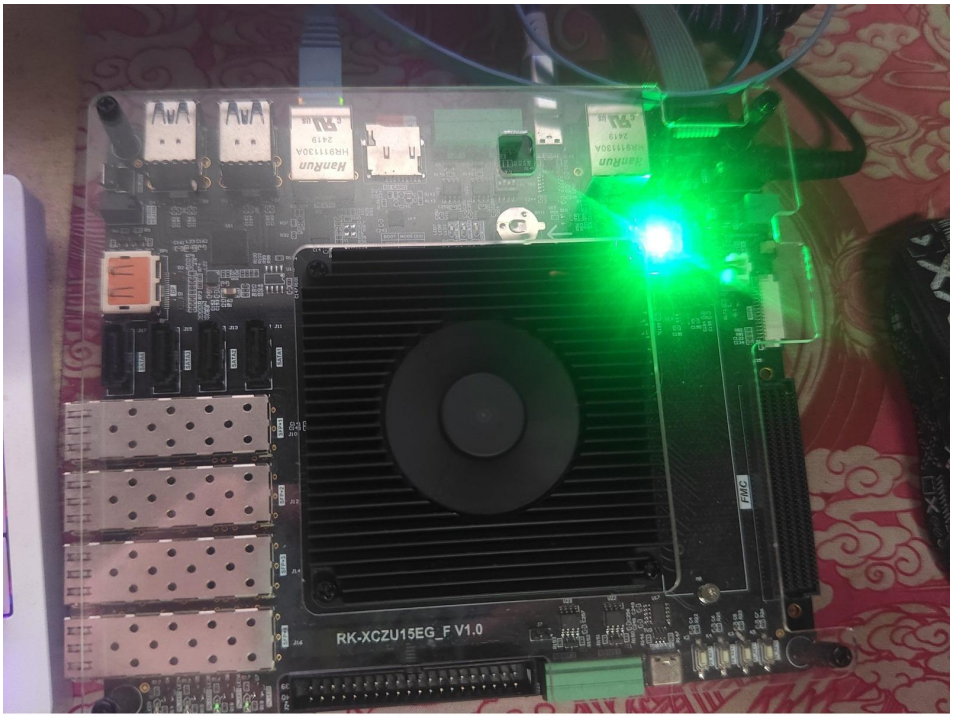


图 8 系统实物照片（RK-XCZU15EG_F 开发板）



图 9 上板调试环境（开发板 + 调试主机）

3.2 工程成果

3.2.1 机械成果：无机械结构件。

3.2.2 电路成果：平台 Block Design、约束文件、位流/HWH/XSA 三件套及哈希配对结果如表 5；哈希核验截图见图 10。

交付物	文件	规模 / 哈希	状态
SNN 平台位流	board/snn_platform.bit + hwh	28,700,853 B；三件套同源配对	已生成（r7，WNS +0.051 / WHS +0.011）
LNN 位流	board/lnn_cfc.bit / .hwh / .xsa	28,700,861 B / 380,358 B / 2,203,972 B	已生成（300 MHz，WNS +0.226 / WHS +0.011）
CNN 历史 r4 平台	platform_top.bit + hwh	历史实现，绑定固定样本板测记录	历史记录，不作当前源码复现结论

① 三件套文件哈希（bit / hwh / xsa）

```
D:\DL_FINAL\LNN\board>E:/miniconda/python.exe -c "import hashlib,os;[print(hashlib.sha256(open(f,'rb').read()).hexdigest
(), f, os.path.getsize(f)) for f in ['lnn_cfc.bit','lnn_cfc.hwh','lnn_cfc.xsa']]"
0fcd8d65d0f8a101d795270741ca7c0cef21f1904590de78226a094b74be30a9 lnn_cfc.bit 28700861
fe2d9f0edcde37d4bfd574d61c079eb892c89de30a052c59d093ad59be6374fb lnn_cfc.hwh 380358
f01d6a8af07fc74acae3dbc30e362db333151978064d3584fcdfc611dbc27032 lnn_cfc.xsa 2203972
```

② XSA 内嵌件哈希：与磁盘文件完全相同 → 配对成立

```
D:\DL_FINAL\LNN\board>E:/miniconda/python.exe -c "import zipfile,hashlib;[print(hashlib.sha256(zipfile.ZipFile('lnn_cfc.
xsa').read(n)).hexdigest(), n) for n in ['lnn_cfc.bit','design_1.hwh']]"
0fcd8d65d0f8a101d795270741ca7c0cef21f1904590de78226a094b74be30a9 lnn_cfc.bit
fe2d9f0edcde37d4bfd574d61c079eb892c89de30a052c59d093ad59be6374fb design_1.hwh
```

图 10 位流/HWH/XSA 三件套哈希配对（XSA 内嵌件与磁盘同哈希）

3.2.3 软件成果：CNN 转译与 IP 打包、三库命令行输出、三库板端界面截图见图 11–图 13。

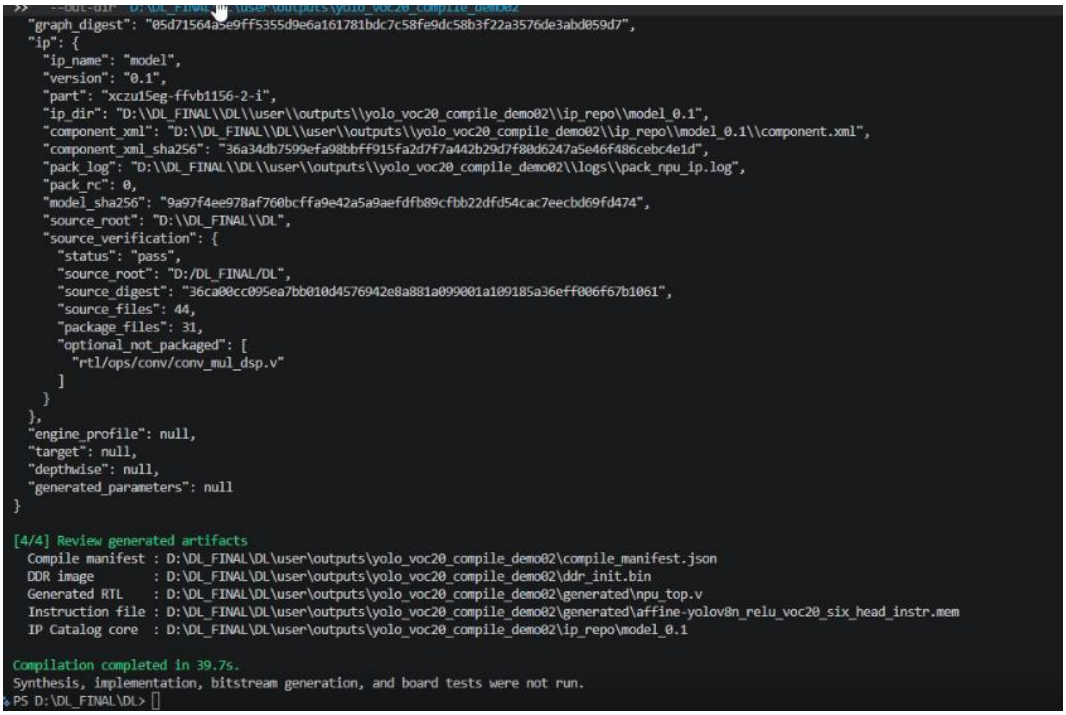


图 11 CNN 转译完成与 IP 打包结果（终端）

CNN：PTH → QDQ ONNX（ONNX Runtime 6 个 head 比对）


```
    },
    "inputs.json": {
      "bytes": 1282,
      "sha256": "99f397fcbeee0528490ea9b83c891e98776a7fb71dcf357ce9fe600282be188e"
    },
    "weights.npz": {
      "bytes": 3116470,
      "sha256": "0b2a8f0b275375bf00e3e383096952aa32080ad546be7fd04592d9553a26ab51"
    },
    "model.onnx": {
      "bytes": 3088698,
      "sha256": "9a97f4ee978af760bcffa9e42a5a9aefdfb89cfbb22dfd54cac7eecbd69fd474"
    }
  }
}

QDQ ONNX: D:\DL_FINAL\DL\user\outputs\yolo_voc20_export_demo02\model\model.onnx
Export report: D:\DL_FINAL\DL\user\outputs\yolo_voc20_export_demo02\model\export_report.json
Source checkpoints copied to: D:\DL_FINAL\DL\user\outputs\yolo_voc20_export_demo02\source_weights
```


SNN：compile 输出（54 参数 blob / 28 份 RTL 绑定）


```
输出  调试控制台  终端  端口
终端                                                                    powershell
PS D:\DL_FINAL\SNN> E:/miniconda/python.exe -B user/show_demo_facts.py --manifest user/outputs/video01/snn/manifest.json --brief
================================================================================
1. compile identity  (source: user/outputs/video01/snn/manifest.json)
================================================================================
model_kind   : real_corpus
structure    : E=128 HEADS=4 HEAD_DIM=32 HIDDEN=512 T=4 TOKENS=32 DEPTH=2
params blobs : 54
rtl sources  : 28  (sha256 bound)
manifest_sha256: f727e4c7920990ab03c11d3b2f26d075...
generated    : adapter, cache_key, header, ooc_tcl, project_tcl, sources_f, xdc
PS D:\DL_FINAL\SNN>
```


LNN：导出重算（7/7 逐字节一致）


```
  SAME tanh_lut.hex              1025 x int16 Q12            5125 bytes
  SAME sigmoid_lut.hex           1025 x int16 Q12            5125 bytes
  SAME gen/cfc_weight_init.svh   RTL weight/bias ROM init    2626 bytes
  SAME gen/cfc_lut_init.svh      RTL activation ROM init     60464 bytes
  SAME gen/cfc_rom_params.svh    N_W / N_B / N_LUT / DT params  115 bytes

task      : same_frequency_sine_delay_4_steps, n_steps=64, delta_t_int=655
metrics   : float task MSE 1.2341e-05 | int vs float MSE 1.1719e-04 | ONNX vs PyTorch max|d| 4.7684e-07

================================================================================
RESULT: PASS  (7/7 files byte-identical)
================================================================================
PS D:\DL_FINAL\LNN>
```


图 12 三库工具链命令输出（上：CNN；中：SNN；下：LNN）

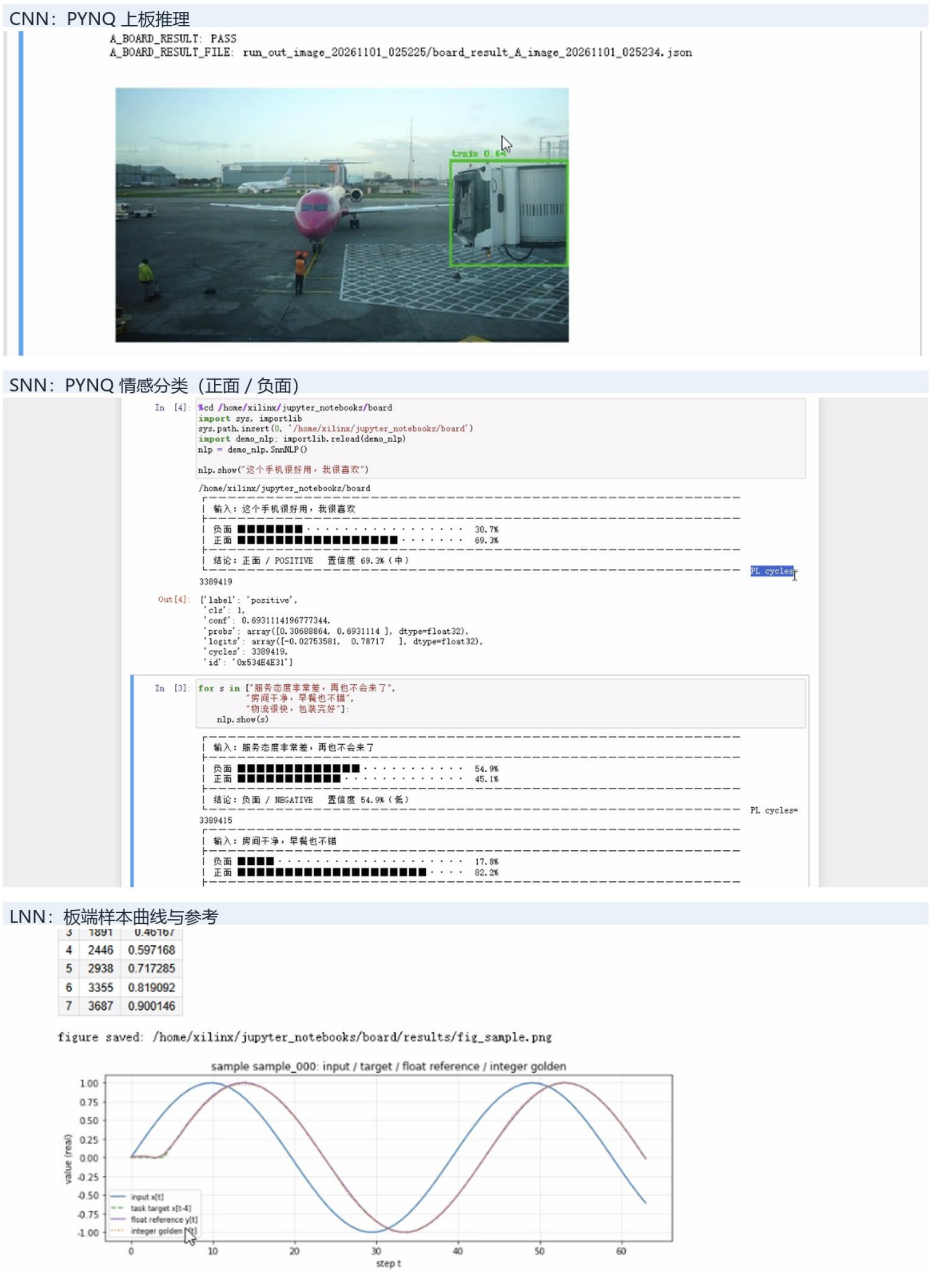


图 13 三库板端运行界面（上：CNN；中：SNN；下：LNN）

3.3 特性成果（量化指标）

表 6 为资源利用率（取自工程记录：CNN 为历史 r4 全平台，SNN 为 5 ns OOC，LNN 为最终位流实现），三库截图见图 14。

库 / 层次	LUT	FF	BRAM	DSP	URAM	器件占比
CNN 历史 r4 全平台	140,028	132,261	157.5	471	72	LUT 41.03% / DSP 13.35%
CNN 旧全算子平台（对照）	191,542	158,733	165.5	527	—	LUT 56.12% / DSP 14.94%
SNN OOC（5 ns）	40,964	71,173	186.5	128	—	LUT 12.0% / DSP 3.6%

库 / 层次	LUT	FF	BRAM	DSP	URAM	器件占比
SNN 平台 r6 资源（r7 未随包）	72,167	110,508	227	131	—	LUT 21.15% / DSP 3.71%
LNN 全设计	25,221	32,890	28.5	4	0	LUT 7.39% / FF 4.82%
LNN CfC 核（单独）	2,189	3,280	—	1	0	单 MAC 时分复用

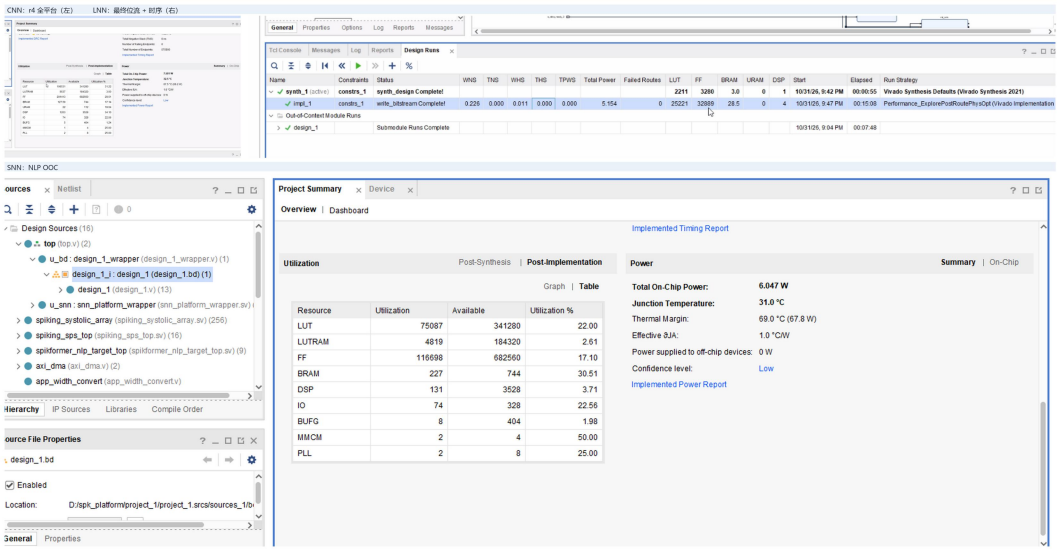


图 14 三库资源利用率截图（上排：CNN / LNN；下排：SNN）

表 7 为时序收敛结果（Vivado 2021.1，均为 0 失败端点）。

对象	约束	WNS / WHS	失败端点	说明
CNN 历史 r4	5 ns	+0.075 / +0.010 ns	0	全平台实现
SNN OOC	5 ns	+0.135 / +0.023 ns	0 / 186,547	routed，0 延迟输入模型
SNN 平台 r7	5 ns	+0.051 / +0.011 ns	0 / 334,330	全片实现并生成位流
LNN	3.33 ns（300 MHz）	+0.226 / +0.011 ns	0	单时钟域，无时序例外

表 8 为功能与精度（每项均指到证据文件）。

指标	CNN 库	SNN 库	LNN 库
功能正确性	ModelSim 15/15（MobileDW）、DW V2 23/23、MaxPool2D 2 快照 0 mismatch	冻结 RTL 回放 4 组 0 mismatch（small 39,774 / e2e 7,084,772 / real 10,627,157 拍）	RTL 逐位仿真 0 mismatch；64 步输出与末状态一致
精度 / 数值	TinyCNN 97.9%（ONNX Runtime）/ MobileDW 63.0%（QDQ 子集）	整数参考 dev 0.7592、holdout 0.7562（多数类 0.5058 / 0.5000）	浮点任务 MSE 1.23e-5；定点 vs 浮点 MSE 1.17e-4
板端状态	历史 8/8 平台 0.2395 s；16/8 平台 51.4 ms（PS 20 + PL 31）	记录级 3/3 匹配（原始 JSON 未随包）	板端离线模型 29/29；真实板测待现场

说明：未执行项一律记为 not_run 或 partial，不将仿真/历史结果表述为本次实测；板端记录与原始日志的差异在报告中并列。

图 15 为 SNN 平台实现摘要（含时序、资源与功耗）；图 16、图 17 为 CNN 16/8 并行度配置的 YOLO 板端运行与资源对照。16/8 平台端到端 51.4 ms，其中 PL 推理约 31 ms、PS 等待约 20 ms；历史 8/8 平台为 239.5 ms，两者不是同一位流，仅作趋势对照。

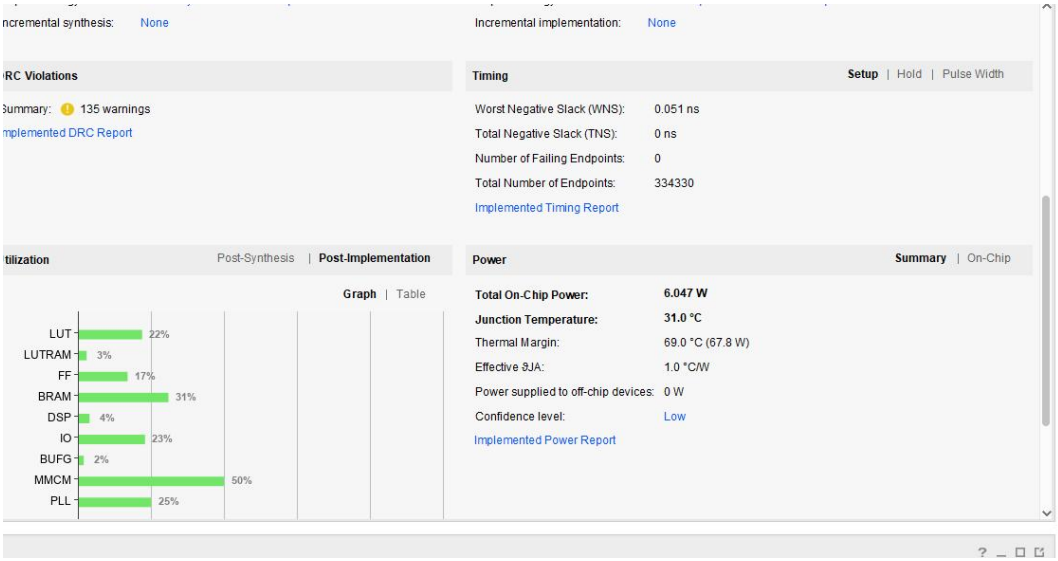


图 15 SNN 平台实现摘要（时序 / 资源 / 功耗）

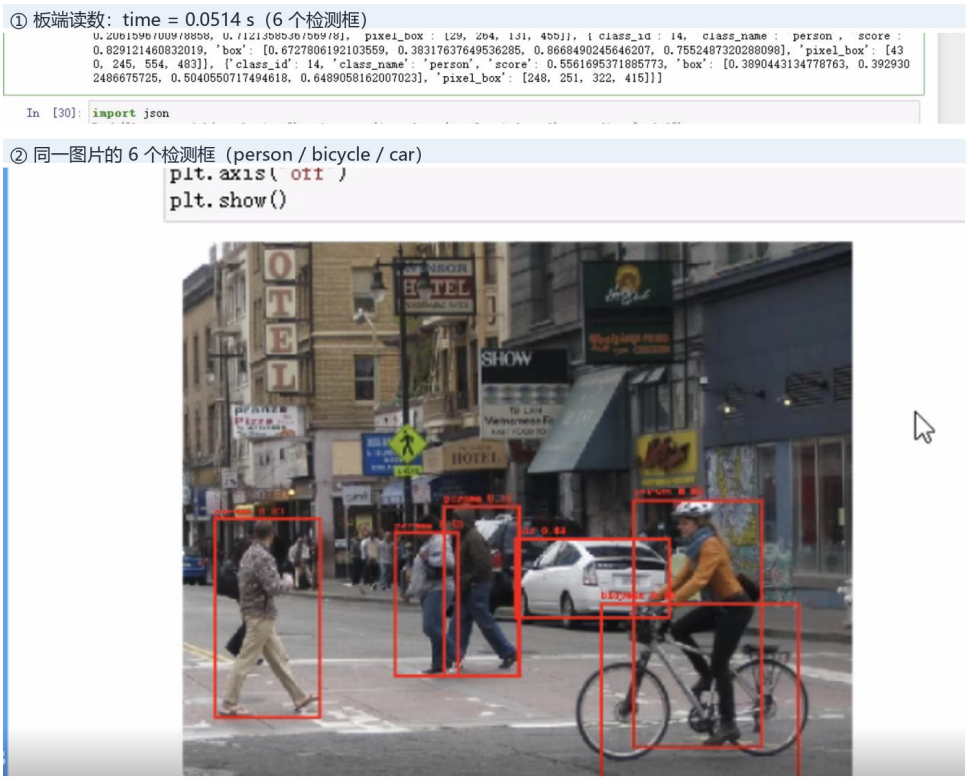


图 16 CNN 16/8 并行度 YOLO 板端运行（端到端 51.4 ms，6 个检测框）

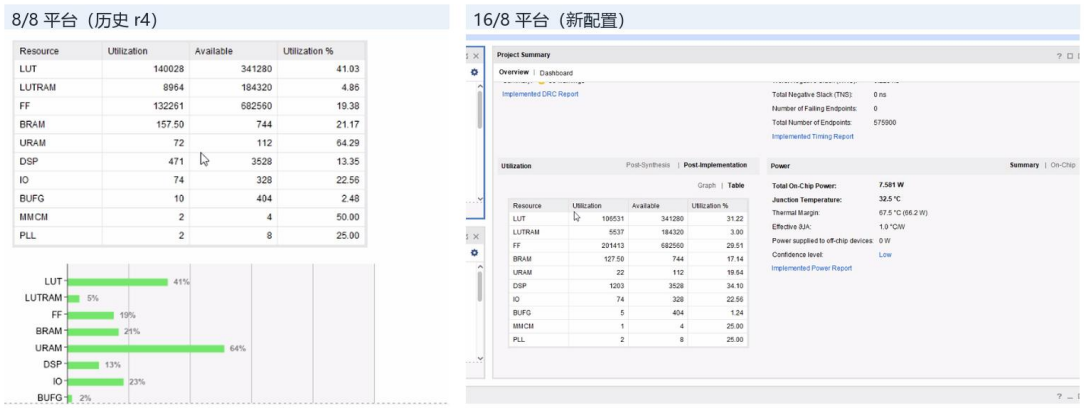


图 17 CNN 并行度资源对照（左：8/8 历史平台；右：16/8 新配置）

第四部分 总结

4.1 可扩展之处

（1）转译器扩展：在现有 ABI 与算子框架下继续增加算子核与量化粒度，覆盖更多检测、分割与轻量 Transformer 模型。

（2）三库协同调度：研究三类模型共享 PL 资源与 DDR 带宽的时分复用调度，在同一平台上串联完成“视觉—文本—控制”任务。

（3）能效与规模：补充功耗测量与能效对比，评估更大模型在更高密度器件上的移植性，并完善多 block 与图像线的整链验证。

（4）开放与复现：将技能包与复现脚本进一步规范化，支持第三方在本仓库上复算证据并扩展到自有板卡。

4.2 心得体会

本作品最特别的地方，是我们把大模型智能体真正放进了工程闭环：智能体负责任务书拆解、代码与脚本实现、证据整理，人负责方向选择、验收口径与关键决策。为了让这种协作可靠，我们逐步形成了几条纪律：一是“事实分级”，每项结论只允许 pass / fail / partial / not_run，每个 PASS 必须指向具体证据文件；二是“任务书与交接”，每次转交都写清目标、边界、必读文档与交付物，避免上下文丢失；三是“证据先行”，先定数值合同与比对方法，再写 RTL 与脚本，最后用同一份参考模型检查三处实现。

过程中也踩过不少坑：例如曾把某个资源门槛当成硬限制反复等待，后来按历史运行数据复核才发现没有依据；曾把核级 OOC 结果当成平台完成，被追问“连 Block Design 都没连”后才补做平台工程；也曾因 Python 缓存文件改变了 manifest 记录的哈希，导致一次板端校验失败。这些教训最后都写进了交接文档与技能包，成为下一轮任务的前置检查项——这也是我们认为“大模型协作”最有价值的部分：不是让模型写更多代码，而是把失败沉淀成可复用的流程与检查表。

回头看不只是做了三个加速器，更重要的是把“怎么让 AI 参与硬件工程”变成可执行的流程：任务书、交接模板、证据索引、技能包，都是可以被下一个人或下一个模型直接复用的资产。我们也清醒地看到不足——图像线与多 block

整链仍未闭环、CNN 主线缺少综合与时序记录、LNN 只支持单一结构尺寸——这些都会写入后续计划，而不是用漂亮数字掩盖。

从工程角度看，三个库共用一套方法的价值在后期体现得最明显：CNN 的 manifest 与 IP 打包思路、SNN 的分级门禁、LNN 的整数参考与寄存器模型，最后都收敛为同一种“模型—数值合同—证据”的表达方式。我们也更清楚地认识到，硬件项目的可复现性不是一句口号，而是由每一次转译、每一份报告、每一个哈希共同保证的；而人机协作真正提升的，是把这些检查变成默认动作的效率。

第五部分 参考文献

[1] Hasani R, Lechner M, Amini A, et al. Closed-form continuous-time neural networks[J]. Nature Machine Intelligence, 2022, 4(11): 992-1003.

[2] Zhou Z, Zhu Y, He C, et al. Spikformer: When Spiking Neural Network Meets Transformer[C]//International Conference on Learning Representations (ICLR). 2023.

[3] Maass W. Networks of spiking neurons: the third generation of neural network models[J]. Neural Networks, 1997, 10(9): 1659-1671.

[4] Fang W, Chen Y, Ding J, et al. SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence[J]. Science Advances, 2023, 9(40): eadi1480.

[5] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8[CP/OL]. 2023. https://github.com/ultralytics/ultralytics.

[6] Jacob B, Kligys S, Chen B, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference[C]//IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 2704-2713.

[7] Howard A G, Zhu M, Chen B, et al. MobileNetV2: Inverted residuals and linear bottlenecks[C]//CVPR. 2018: 4510-4520.

[8] Open Neural Network Exchange (ONNX) Specification[EB/OL]. https://onnx.ai/onnx/.

[9] AMD/Xilinx. Vivado Design Suite User Guide: Synthesis (UG901)[Z]. 2021.

[10] AMD/Xilinx. Vivado Design Suite User Guide: Implementation (UG904)[Z]. 2021.

[11] AMD/Xilinx. UltraFast Design Methodology Guide for FPGAs and SoCs (UG949)[Z]. 2021.

[12] AMD/Xilinx. Zynq UltraScale+ MPSoC Technical Reference Manual (UG1085)[Z]. 2021.

[13] PYNQ Project. PYNQ: Python Productivity for Zynq[EB/OL]. https://pynq.readthedocs.io/.

[14] LeCun Y, Bengio Y, Hinton G. Deep learning[J]. Nature, 2015, 521(7553): 436-444.